
CEREBYL · INTERNAL ENGINEERING REVIEW

WhatsApp Integration: Build Review & Feature Plan

What we have built, what is actually working, what is quietly broken, and how WhatsApp should connect to the rest of the app.

Prepared for: Harish Sharma

Date: 16 August 2026 · *revision 2 — adds Part 5 (owner controls & rep access)*

Method: Full read of `cerebyl-whatsapp-worker` (13 files), 6 Supabase edge functions, 6 migrations, and the `/whatsapp` app surface (1,767-line route + 7 components). No code was modified.

Policy basis: Meta WhatsApp Business Platform & Commerce Policy, India 2026 pricing.

Contents

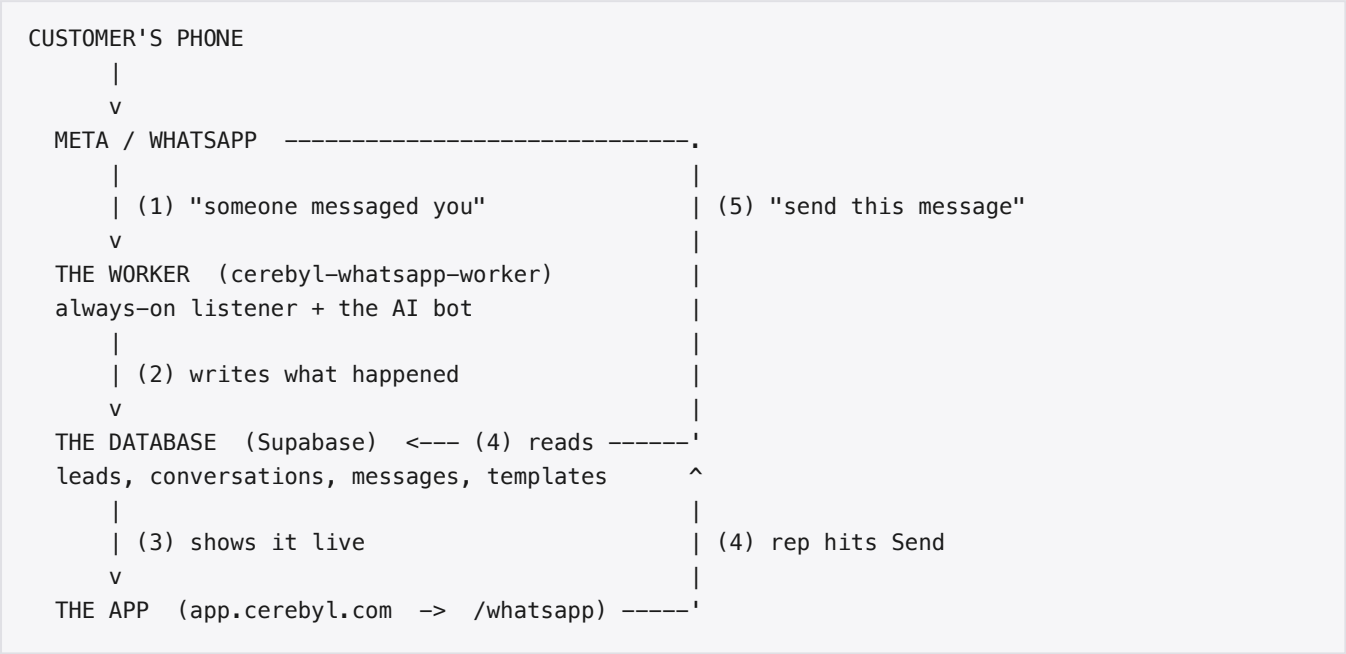
1. **Part 1 — What we have built, explained simply.** Every moving part in plain language.
2. **Part 2 — The health check.** What is correct, and 15 things that are wrong, ranked.
3. **Part 3 — How WhatsApp should connect to the app.** The architecture and the reasoning.
4. **Part 4 — New features.** What to build, why it matters, and what it costs.
5. **Part 5 — Company controls & rep access.** The owner's switchboard and the rep permission model.
6. **Part 6 — The order to build in.**

The one-paragraph summary

The engineering underneath this integration is genuinely good — the webhook security, the duplicate-message handling, the race conditions, the fallback chains and the MRP-only discipline are all done properly, better than most commercial WhatsApp tools. **The problem is at the two edges: the dashboards and the compliance boundary.** Three screens report information they do not actually have (number health, delivery stats, the AI knowledge base), and three compliance mechanisms have holes that could cost a client their WhatsApp number. None of it is hard to fix. All of it should be fixed before a second client is onboarded.

Part 1 — What we have built, explained simply

There are three separate programs involved, running in three different places. Most confusion about this system comes from not knowing which one does what.



1. The Worker — the part that never sleeps

A small program living on Cloudflare's network. Meta needs somewhere to deliver messages the instant they arrive, 24/7, and a website cannot do that (a website only runs when someone opens it). The Worker is that always-on address. It does four jobs:

- **It checks the message is really from Meta.** Every incoming message carries a cryptographic signature. We verify it before reading a single word — so nobody can fake a customer message into your CRM. (This is done correctly.)
- **It turns a message into a lead.** New number writes in → the Worker creates a conversation, creates a lead, assigns a rep by round-robin, and logs the message. The rep gets a notification through the normal app pipeline — no separate alert system was needed.
- **It runs the AI bot.** Reads the conversation, the lead record and the company's product catalogue, then replies as a salesperson.
- **It records delivery ticks.** Meta tells us sent → delivered → read; the Worker files those against the message.

2. The bot's brain, in plain terms

The bot is Google's Gemini model, given a personality, a memory, and exactly four buttons it can press. The four buttons are the important part — **the bot physically cannot do anything outside them**, no matter what a customer asks or how they phrase it.

The button	What it does
------------	--------------

<code>update_lead_details</code>	Writes what it learned into the lead record — name, firm, profession, city, state, DL/GST status, and free-text notes that accumulate rather than overwrite.
<code>share_product_list</code>	Asks a separate program to build a branded PDF catalogue and sends it as a document. MRP only — trade rates, PTS and PTR are never even loaded into the bot's memory, so it cannot leak them by accident.
<code>share_product_images</code>	Sends up to 3 product photos with MRP captions. Honestly reports back which products had no photo, so it doesn't invent one.
<code>mark_ready_for_handoff</code>	Says "a rep will call you", then steps back.

Its instructions tell it to text like a real salesperson: one or two lines, never a paragraph; acknowledge before acting ("ok", "hanji", "theek hai"); ask for the customer's preferred language on the first message and stay in it; and a **two-strike rule** — ask for any missing detail at most twice, then drop it gracefully rather than nagging. It can genuinely see photos, hear voice notes and read PDFs.

3. The app — the five tabs on `/whatsapp`

Tab	What it is for	State
Inbox	Live chat threads, updating in real time. Shows a countdown of the 24-hour free-reply window, lets a human take over from the bot, and attach files.	Working well
Templates	Write a pre-approved message, submit it to Meta, check if it was approved. Has an AI generator that writes the template for you from a plain-Hindi prompt.	Working, with gaps
Broadcasts	Send an approved template to many leads or parties at once.	Works, but reports false numbers
Knowledge	Teach the bot your company's schemes and policies.	Not connected to the bot
Health	Show the number's quality rating and sending tier.	Shows fixed placeholder values

4. The two rules Meta enforces, which shape everything

The 24-hour window. When a customer messages you, a 24-hour window opens in which you can reply with anything — free text, photos, PDFs. Once it closes, you may only send a *pre-approved template*. This is why templates exist at all, and why the Inbox shows a countdown. The app enforces this on the server, not just in the interface. That is correct.

Cost, India 2026. Marketing message ≈ ₹0.88. Utility message (order dispatched, payment due) ≈ ₹0.115. That is a **7.6× difference** for what is often the same sentence in a different category. Getting the category right is not paperwork — it is the difference between a ₹1,000 and a ₹7,600 monthly bill. Note also that free in-window replies **stop being free on 1 October 2026**.

Part 2 — The health check

First, what is genuinely well built

This deserves saying plainly, because the list of problems below is long and would give a false impression on its own. The following are done to a standard I would not expect at this stage:

- **Webhook signature verification** before the payload is parsed — no forged messages.
- **Duplicate protection.** Meta delivers the same message twice by design. A unique database index makes the second copy fail, and the code swallows that failure quietly. Without this, *every customer would get every bot reply twice.*
- **The two-messages-at-once race is handled.** If a new customer sends two messages in the same second, both could create a conversation. A partial unique index makes the loser fail, and the code re-fetches the winner's row instead of creating a duplicate thread.
- **Delivery ticks only ever move forwards.** Meta's callbacks arrive out of order; the code refuses to downgrade "read" back to "sent", and treats "failed" as permanent.
- **MRP discipline is structural, not hopeful.** The catalogue given to the AI only ever contains the MRP column. There is no path by which a trade rate could be spoken, even if the model tried.
- **The AI's category answers are validated** against the exact list the database allows, so the bot cannot write a value that silently fails.
- **The customer is never left in silence.** Cached prompt → plain prompt → retry → and finally a safe generic holding message. Four layers deep.
- **Handoff only commits if the message actually sent.** A failed send leaves the conversation with the bot so the next message retries, rather than stranding the customer.
- **Permissions are tight.** Column-level database grants (a rep can flip a status but cannot rewrite who owns a conversation), and reps only see their own threads.

Now the problems, ranked

CRITICAL 1. The Health tab is decorative — it shows green no matter what

In plain terms: the screen that is supposed to warn you that a client's WhatsApp number is in trouble is not connected to anything. It displays "GREEN" and "TIER_1K" as fixed text. Nothing in the entire system ever asks Meta what the real quality rating is, and nothing ever writes it to the database — the column exists and is permanently empty.

Why it is worse than having no screen at all: on the day a client's number goes Red and their messages stop reaching customers, this panel will still say everything is fine. You would find out from an angry phone call. *A dashboard that cannot be wrong is not a dashboard.*

CRITICAL 2. The AI Knowledge Base does not reach the bot

In plain terms: a company admin fills in their schemes, territory policy and sales rules, clicks Save, and gets a confirmation reading *"Bot will now answer prospective distributors according to your updated company policies."* The bot never reads any of it. The text is saved into a settings field that the bot does not load — it only pulls address, phone, email and website.

So the feature looks complete, tells the user it worked, and changes nothing about how the bot behaves. This is the most misleading thing in the build.

CRITICAL 3. "STOP" is ignored once the bot has handed off

In plain terms: the code that catches a customer typing STOP or UNSUBSCRIBE sits *after* the check that makes the bot go quiet post-handoff. Since handoff is the goal of nearly every conversation, this means: **for most customers, replying STOP does nothing.** No opt-out is recorded, no confirmation is sent, and they stay in every future broadcast.

Why it matters beyond politeness: honouring opt-out is a Meta requirement and an Indian DPDP requirement. Repeated ignored opt-outs are exactly what drives a number's quality rating down and gets a WhatsApp Business account suspended.

CRITICAL 4. Broadcast reports "delivered" for messages it merely handed over

In plain terms: when the broadcast sends a message and Meta replies "OK, I've got it", we record that recipient as **delivered**. But "OK, I've got it" is not delivery — it means accepted for delivery. The phone may be off, the number may be invalid, the customer may have blocked you.

Worse, nothing ever updates those rows afterwards. Real delivery confirmations arrive later by webhook, but the code that handles them only looks at the inbox message table — and broadcast messages are never written there. So **campaign statistics will read close to 100% delivered, permanently, regardless of reality**, and broadcast messages are invisible in the Inbox.

CRITICAL 5. Broadcasts silently stop at 500 recipients

In plain terms: the audience query has a hard cap of 500 rows and no paging. A company with 900 distributors sends to 500 of them, and the report says "targeted: 500" — so it looks correct. Nobody would ever notice.

This is exactly the failure mode `fetch-all.ts` was written to eliminate everywhere else in the app. That safeguard was never applied here.

HIGH 6. Seven places send WhatsApp messages; only one checks opt-outs

In plain terms: messages leave through seven different pieces of code across three codebases. Each one has its own opinion about the rules. Only the broadcast sender checks whether the customer has opted out — **a rep replying by hand, and the bot itself, will happily message someone who said STOP.**

This is the root cause of several other items on this list, and the strongest argument for the architecture change in Part 3.

HIGH 7. Meta never tells us when a template is approved

Meta pushes template approvals, rejections and quality warnings to a webhook. We only subscribe to messages and delivery ticks. So an approval only appears when a human opens the Templates tab and clicks Refresh on that one template. And **template quality warnings — the ones that precede a template being permanently disabled — are invisible to us entirely.**

Meta's pause ladder: first low-quality strike = 3 hours paused, second = 6 hours, third = permanently disabled. We currently cannot see any of the three.

HIGH 8. Anyone on the internet can spend our AI budget

The Worker's template-generator endpoint has no authentication and accepts requests from any origin. It runs on the *platform* Gemini key — our money, not the client's. Anyone who discovers the URL can generate unlimited templates at our expense.

HIGH 9. There is no opt-in record at all

We track who opted *out*. We have nothing recording who ever agreed to receive marketing. Meta requires demonstrable opt-in for marketing messages; DPDP requires recorded consent with its source. Today, **every lead and party in the database is treated as broadcastable by default** — including a number scraped from a directory or typed in by a rep.

Related: opt-out records can be *deleted* by any manager, because the permission rule grants full control rather than insert-and-read. A record of someone withdrawing consent should not be erasable.

HIGH 10. Your existing customers are treated as strangers

In plain terms: when an inbound message arrives, we look for a matching *lead*. We never look for a matching *party* (an actual customer). So a distributor you have been invoicing for two years messages the company number, and the system creates a brand-new lead, round-robins it to whichever rep is next, and the bot opens with "*May I know your good name?*"

The database even has the field to link a conversation to a party. Nothing ever fills it in. For a PCD business where the same distributors message constantly, this is the single most visible product flaw on the list.

HIGH 11. Broadcasts ignore Meta's speed and frequency limits

The sender fires roughly 50 messages per second and never asks what the number's tier allows. It records failures but does not slow down or stop when it starts hitting Meta's per-user frequency cap (a business may send only about **2 marketing messages per person per 24 hours** unless they reply). Hammering through those errors is precisely how a quality rating collapses.

HIGH 12. Two copies of the template-writing AI, already drifting

The same system prompt exists in the Worker and again inside the edge function, written out separately. They are already slightly different. This project has been bitten by duplicated logic three times before — the fallback in one of them is the next item.

HIGH 13. The emergency fallback template makes claims on your behalf

When the AI generator fails, a hardcoded template is used instead. Its text promises a "*DCGI approved price list*" and a "*15% margin*". That fires unattended, for whichever company happens to be using it, asserting a regulatory approval and a commercial term that may be untrue. A fallback should be bland, not confident.

MEDIUM 14. Products are capped at 400 and cached for an hour

The bot loads at most 400 products. A company with 600 has 200 products the bot has never heard of, and will say so to a customer. Separately, the catalogue is cached for one hour with no way to clear it — **so a price correction can take up to an hour to reach a live conversation**. This is the same caching trap already documented elsewhere in the project, reappearing in a new place.

MEDIUM 15. Smaller items

- The bot's live knowledge of the lead is attached to the *oldest* message in the conversation rather than the newest, so on a long chat the freshest facts sit furthest from the model's attention.

- Saving the Knowledge Base can silently affect zero rows while still showing a success message (it filters on a field that is not where the company id actually lives). Contained by permissions, so not a security issue — but a silent failure.
- A later migration re-granted reps the ability to rewrite which lead a conversation points at, partly undoing the tightening done the day before.
- Nothing anywhere records what a message cost. With a 7.6× price gap between categories, neither you nor the client can see where the money went.

The pattern worth naming

Every critical item is the same shape: **a screen that reports something it does not actually measure.** Health says green without asking. Broadcast says delivered without checking. Knowledge says the bot learned without telling it. The plumbing is careful and defensive; the instrumentation is optimistic. That is a fixable habit, and the fix is a rule: *a number on a screen must come from a measurement, or it must not be on the screen.*

Part 3 — How WhatsApp should connect to the app

The question underneath everything

There are two ways to build this, and we are currently drifting toward the wrong one.

Option A — WhatsApp as a section	Option B — WhatsApp as a transport
WhatsApp gets its own page with its own inbox, its own contacts, its own templates and its own audiences. It sits <i>beside</i> the CRM.	WhatsApp is a way for existing records to talk. A lead has a WhatsApp thread. A party has one. An order sends its own status update. There is no "WhatsApp section" for a rep — only for the manager who configures it.
This is what BotBiz is. It is also what a 1,767-line <code>/whatsapp</code> route with five tabs becomes if we keep going.	This is what a CRM with WhatsApp should be, and it is our actual advantage over BotBiz.

The test that settles it: a rep should never have to navigate to "the WhatsApp page" to do their job. They should message a customer from the customer's own page. If they must go somewhere else, we have built a second product inside the first one, and the two will slowly disagree about who the customer is.

Principle 1 — One identity per phone number

Today, an inbound number is checked against leads only. It should pass through a single resolution step before anything else happens, and every part of the system should use that one answer:

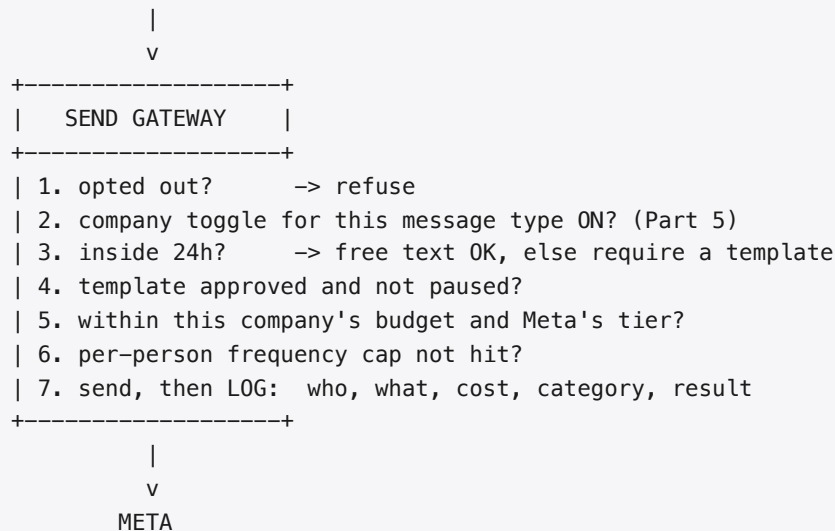
```
inbound phone number
|
v
[ RESOLVE ] -- is this a PARTY?  -> existing customer: skip qualification entirely,
|                                     greet by name, offer reorder / statement / order status
|                                     -- is this a LEAD?    -> continue the qualification already in progress
|                                     -- is this STAFF?    -> internal, do not treat as a customer
|                                     -- nobody?          -> create a lead, assign a rep, start qualifying
v
one conversation, correctly attached
```

This one change fixes problem 10, prevents duplicate customer records, and unlocks half the features in Part 4 — because a bot that knows it is talking to an existing distributor can answer "where is my order" and "what do I owe", which a bot talking to an anonymous number never can.

Principle 2 — One door out

Right now seven pieces of code call Meta's send endpoint directly, each with its own view of the rules. That is why opt-outs are enforced in one place and ignored in six. Everything outbound should pass through a single gateway that applies the rules once:

bot reply · rep's manual reply · broadcast · order-status update · follow-up nudge



Why this is the highest-value change in the document: every compliance rule, every cost control, every rate limit and every piece of reporting becomes one piece of code in one place. Today, adding a new rule means remembering to add it in seven places, and we have already proven we will forget.

Principle 3 — Events go out; screens do not come in

The CRM already knows when an order is dispatched, when a payment is overdue, when a follow-up is due. Those events should be published into the same gateway, so the message sends itself. The alternative — a rep remembering to open WhatsApp and type it — is the chore work we are trying to remove.

Practically: reuse the notification-generator pattern already in the app (the scheduled functions that produce due reminders). Do not build a second scheduler.

Principle 4 — The CRM is the memory; the AI is not

This is already done correctly and should be protected. The bot writes what it learns into the lead record and reads it back next turn. It does not keep its own private memory. That means a rep opening the lead sees everything, the AI can be swapped or restarted with nothing lost, and there is one version of the truth. *Do not let a future feature keep state only inside the AI.*

Principle 5 — A cache must be cleared by an event, not by a timer

The product catalogue is cached for an hour. When a company changes a price, that change should clear the cache immediately, not wait out the clock. Same rule for the knowledge base once it is wired up. A stale price quoted to a distributor is a real commercial problem, not a technical one.

Principle 6 — Never surface a number we did not measure

Covered above, but it belongs in the architecture list because it is a build rule, not a bug fix. If a panel cannot get real data yet, it should say "not connected" — never a plausible placeholder.

A note on what we must NOT build

WhatsApp's Commerce Policy prohibits selling pharmaceuticals. Meta bars "promoting or otherwise facilitating the exchange of prescription drugs", and bars commerce content for medical and healthcare products. Pharmacies get a narrow carve-out for grocery and convenience items; for OTC drugs they may use *messaging only*. Manufacturers may sell services, never product.

What this means for us: WhatsApp's own Catalog, Cart and in-chat Payments are off the table permanently — and so is the "catalog orders" feature copied from BotBiz. *Talking* about products is fine, which is what our bot does. Taking the order must happen in Cerebyl, with WhatsApp only carrying the conversation and the confirmation. This is commercially good news: the transaction stays in our product rather than Meta's.

Part 4 — New features

Tier 1 — Fix what is claimed but not true

These are not new features. They are the difference between a demo and a product. Each one is currently on screen and lying.

1.1 Real number health

Ask Meta, on a schedule, for each number's actual quality rating, messaging tier, verification status and marketing eligibility. Store it. Alert the moment it moves. **Why it matters:** losing a WhatsApp Business account is an extinction event for a distributor business. Days of warning is the most valuable thing we can give a client, and Meta hands us the data for free.

1.2 Real broadcast reporting

Record what Meta actually said (accepted), then update each recipient as delivery and read confirmations arrive by webhook. Show the true funnel: *targeted* → *accepted* → *delivered* → *read* → *failed*, with failure reasons in plain English rather than error codes. Remove the 500-row cap and page the audience properly.

1.3 Connect the Knowledge Base

Load the company's schemes, territory rules, dispatch times and MOQ into the bot's instructions, and clear its cache when they change. Today the bot deflects most policy questions to a human because it genuinely has no source. This is what turns it from a form-filler into something that answers.

1.4 Fix opt-out, and add opt-in

Catch STOP before the bot goes quiet, in every language a customer might use it. Make opt-out records permanent rather than deletable. Then add the missing half: a consent record showing who agreed to marketing, when, and how — so a broadcast audience can be built from people who actually said yes, not from everyone in the database.

1.5 One send gateway

The consolidation described in Part 3. Everything else in this list gets easier once it exists, and several items are not safely buildable without it.

Tier 2 — Remove the chore work

2.1 Template autopilot

Today a rep writes a template, guesses a category, submits it, and may be rejected for a reason they cannot interpret. Instead: generate the whole template pack for a company at onboarding from their own product and scheme data; **check it against Meta's rules before submitting** (claims, category mismatch, variable placement, missing examples, missing opt-out); submit; catch the verdict on the webhook rather than making someone click Refresh; and on rejection, rewrite and resubmit automatically, escalating to a human after two failed attempts.

The pre-submission check is the valuable half. It is also where the category discipline lives — and category is a 7.6× cost multiplier.

2.2 Language fan-out

One approved English template automatically translated and submitted for Hindi, Punjabi, Gujarati, Marathi. Meta groups templates by name and language, so the send layer can then pick automatically based on the language the customer has been writing in — which the bot already detects.

2.3 Order lifecycle messages

Placed → packed → dispatched → delivered, sent automatically with the tracking link the app already generates. These are **Utility** templates at ₹0.115, and they are the highest-trust message a distributor ever receives from a supplier. This is the cheapest, highest-return feature in the entire document. (Subject to the company's toggle — see Part 5.)

2.4 Buttons and list menus

The bot is text-only today. WhatsApp supports tappable buttons and menus. For a distributor typing on a phone this cuts the conversation length dramatically — which also cuts our AI cost, because shorter conversations mean fewer tokens.

2.5 Cost meter and budget cap

Show each company its live WhatsApp spend split by category, with a monthly ceiling that stops sending and alerts rather than surprising them with a bill. Essential before 1 October 2026, when in-window replies stop being free.

2.6 Follow-up sequences

Wire timed nudges to the follow-up slots the CRM already has, so a lead going quiet gets a message without a rep remembering. Reuse the existing scheduled-notification machinery.

Tier 3 — Make the rep look advanced

3.1 The bot serves existing customers

Once phone numbers resolve to parties (Part 3, Principle 1), the bot can answer "where is my order", "what is my outstanding", "send my statement" and "same as last time" — with everything scoped so a customer only ever sees their own data. This is the moment WhatsApp stops being a lead-capture channel and becomes a service channel. (Every one of these is individually switchable — Part 5.)

3.2 Ambient order capture

A distributor sends a photo of a handwritten order pad, or a rambling voice note. The system parses it into line items, checks them against stock and that customer's rate card, and returns a proforma PDF for confirmation. The rep sees a finished order rather than a chat to read. The bot already understands photos and voice — this is mostly plumbing to the order system.

3.3 Payment intimation capture

Customer sends a UTR or a payment screenshot; the system reads it and files a *pending* record for a human to confirm. It must never post a payment on its own — that rule already exists in the distributor portal and should be reused, not reinvented.

3.4 Two-way voice

The bot already understands voice notes. Let it reply in voice, in Hindi. A large share of PCD distributors would rather talk than type, and no CRM in this segment does this.

3.5 Conversation intelligence

Automatic thread summaries, buying-intent scoring and objection tagging, feeding the existing lead score — so the pipeline reflects what was actually said, not what a rep remembered to type.

3.6 Rep response dashboard

First-response times, threads left unanswered, handoff-to-close rate. Turns the inbox from a place work happens into an instrument a manager can run the team with.

Tier 4 — The differentiators

4.1 Ban-risk forecasting

Once real health data exists (1.1), model each number's trajectory — quality, block rate, read rate, send mix, frequency-cap hits — and warn *days before* Meta acts, naming the campaign to pause. This is the most valuable single thing we could offer a client, and it is a natural extension of work already required.

4.2 Self-optimising templates

Measure read rate, reply rate and conversion per template. Write variants, promote what works, retire what does not, and rewrite anything drifting toward a quality pause before Meta disables it. The company's message library improves on its own.

4.3 Cashflow-aware follow-up

The system already knows every distributor's dues, ageing and payment habit. Let it choose *when* and *how firmly* to chase each one, throttling itself to protect both the relationship and the number's quality rating. Collections that behave like a good human, across hundreds of accounts.

4.4 Company-wide conversation memory

Ask the in-app assistant: "*which distributors complained about packaging last quarter?*" — answered from every conversation, every rep, every number. Institutional memory that survives a rep resigning.

4.5 Ten-minute onboarding

A new client connects their WhatsApp account and, within ten minutes, the system has read their product master, generated and submitted their template pack in five languages, built their knowledge base and gone live. Competitors bill a week for "setup".

4.6 Compliance evidence pack

Every message pre-screened against Meta's policy, Indian drug-advertising law and each contact's consent state — plus a one-click export if Meta or a regulator ever asks. Compliance stops being a worry and becomes something to sell on.

Part 5 — Company controls & rep access

This part designs out three requirements from the owner: a granular switchboard so a company decides how much WhatsApp gives away, a permission model that keeps reps off the company number by default, and a way to stop the bot the moment a rep takes over.

5A. The capability switchboard

The strategic point, stated first

This is not a settings screen — it is **channel strategy**. Cerebyl sells branded mobile apps and a distributor portal. If WhatsApp answers every question a distributor has, there is no reason for that distributor to ever install the company's app. Some owners will want WhatsApp to be a wide-open service desk; others will want it deliberately narrow, so that order status and statements are things you open the app for. **Both are legitimate, and the product should not choose for them.** The switchboard is how an owner sets where that line sits.

Three rules for how the toggles must be enforced

1. **Enforce by removing the tool, never by instructing the AI.** If "order status" is off, delete that capability from the bot's tool list for that company. Telling the model in its prompt "do not discuss order status" works most of the time, and most of the time is not a guarantee. A tool that does not exist cannot be called. *This is the principle the bot already relies on today — its four buttons are its hard ceiling — and it should be how every toggle works.*
2. **Every "off" needs a redirection, not a refusal.** A toggle set to off should not produce "I can't help with that". It should produce "*You can see that live in our app — here's the link*". That turns each switch from a restriction into an app-adoption funnel, which is the actual reason the owner turned it off.
3. **The same toggle is checked at the send gateway.** Otherwise switching off "order status" stops the bot answering it but does not stop the scheduled dispatch message going out. One switch, both directions.

The proposed switch list

Group	Switches	Default
Master	WhatsApp bot enabled. Off = messages are still received, still logged, still create leads — but the bot never replies and a human handles everything. Capture and conversation are separate concerns; turning off the bot must never turn off lead capture.	On
Prospects (new leads)	Qualify new leads · Share product list PDF · Share product images · Auto-detect and mirror language · Auto-handoff to rep when qualified	All on
Existing customers (distributors / parties)	Serve existing customers at all · Order status & tracking · Outstanding dues · Account statement · Reorder / repeat last order · Payment intimation capture · Product & price queries	Serve = on; the rest off until the app-vs-WhatsApp choice is made deliberately

Automatic outbound	Order placed confirmation · Packed · Dispatched (+ tracking) · Delivered · Payment received receipt · Payment due reminder · Follow-up nudges · Scheme & launch broadcasts	Off every one of these costs money per send
Redirection mode per switch	Stay silent · "Please check the app" + deep link · "Please contact your sales associate" + rep name and number	App link where an app exists

Two honest cautions before this ships

1. The app cannot yet replace what you would switch off. Turning off WhatsApp order updates in favour of app notifications assumes the app can reach a distributor whose app is closed. It currently cannot — phone notifications are at stage 1 (the phone schedules them itself, which only works while the app has been opened recently), and true push needs Firebase work that has to be repeated for every branded per-company build. **An owner switching to "app only" today would quietly get a worse experience, not a better one.** Either finish push first, or say this plainly in the toggle's help text.

2. Order-status messages are the cheapest, most trusted thing we send. They are Utility templates at ₹0.115, and a dispatch notification is the single message a distributor most wants to receive. Switching them off to drive app installs is a real trade, not a free one. It is the owner's call — but the interface should tell them what they are giving up rather than presenting it as a neutral preference.

Where it should live: the app already has a feature-flag system with the right shape — a list of keys, a set that defaults off, and a set only the platform console may change. Extend that with a third tier, *company-owner-controlled*, rather than inventing a parallel settings system. A second flag mechanism is how two parts of an app start disagreeing about whether a feature is on.

5B. What reps can and cannot do

Four of the six things asked for here are partly built already, and one is currently the wrong way round. Checking before building is the rule on this project, so here is the honest position:

What you asked for	Where it actually stands
A rep sees only their own leads' chats, never another rep's	DONE Already enforced at the database level. A rep physically cannot read another rep's conversation or its messages — it is the same rule the rest of the app uses for leads.
A rep is read-only on the company's main number by default	NOT TRUE Reps <i>can</i> send from the company number today, on any conversation assigned to them. This needs building.
An admin toggle to grant reps sending rights on the main number	NEW
A tick — "have you contacted this lead?" — that stops the bot	NEW though the pieces exist: the lead pipeline already has a "Contacted" stage, and reps already have permission to change a conversation's state.

After that, the number politely redirects the lead to the rep

NEW — and it fixes something worse than a missing feature. Today when the bot hands off it simply *goes silent*. A lead who keeps messaging gets nothing back at all. Silence is a worse experience than the redirect you are asking for.

The lead record and the WhatsApp chat are merged so a rep sees everything in one place

BACKWARDS A panel exists that shows the *lead* inside the WhatsApp inbox. Nothing shows the *WhatsApp conversation* inside the lead page — and the lead page is where a rep actually lives.

Read-only by default, with an owner override

A company setting — "Allow reps to reply from the company number", default **off**. Enforced on the server, not just by hiding the button, because a hidden button is not a permission.

When it is off, the rep sees the full conversation, and the message box is replaced with a clear line and a one-tap action:

Replies from the company number are restricted. Message this lead from your own number. → [Open in WhatsApp]

That button opens the rep's own WhatsApp with the lead already addressed, so the restriction costs them nothing in effort. Without it, the rule just feels like an obstacle and reps will work around it.

The cost of this design, stated plainly: anything a rep sends from their personal phone is *invisible to the CRM*. No transcript, no delivery ticks, no searchable history, and nothing for the AI summary to read. The real answer for reps who matter is to connect **their number to the platform too** — the system already supports per-rep numbers, and the bot already knows how to hand a conversation across to one. Then the rep messages from their own number *and* everything is still logged. Recommend that as the paid-tier path; treat personal-phone messaging as the fallback for small teams.

The "I've contacted them" tick, and stopping the bot

This is the API-cost control, and the design matters more than it looks.

- **Where the tick lives:** three places, same action — on the lead card in the new-leads list, in the WhatsApp thread header, and on the lead detail page. A rep should never have to hunt for it.
- **It must override the "always on" setting.** A company can currently set the bot to keep talking forever. If the rep's tick does not beat that setting, this feature will silently do nothing for exactly the companies that need it most. Order of authority: *rep has taken over* > *company always-on* > *default*.
- **No AI call afterwards — that is where the saving comes from.** Once ticked, further messages from that lead get one fixed, warm, pre-written line naming the rep and their number. The language model is never invoked again for that conversation. The cost goes to zero, not down.
- **Send it at most once per day.** A chatty lead must not receive the same redirect fifteen times — that is the kind of thing that gets a number reported.
- **Word it as a graduation, not a brush-off.** Something like: "Aapki enquiry [Rep] ko de di gayi hai — wo aapko [number] se direct contact karenge. Aap unhe seedha message kar sakte hain." The lead should feel promoted to a human, not dismissed by a robot. Wording should be editable per company and follow the language the lead was already using.

- **One source of truth.** Store it on the conversation. Moving the lead's stage to "Contacted" may set it as a side effect — but never the reverse. Two fields that update each other in both directions always drift apart eventually.

The merged lead ↔ WhatsApp view

The rep's home is the lead page. Everything the bot learned should be there, not behind a separate WhatsApp section. The panel should carry:

1. **An AI brief** — what they asked for, what they actually want, any objection raised, and the obvious next step. Three or four lines a rep can read before dialling.
2. **The full transcript, read-only**, including the photos, voice notes and PDFs the lead sent.
3. **The extracted fields** — firm, profession, area, DL/GST, product interest — with a marker showing which the bot filled in versus which a human did.
4. **The "I've contacted them" tick** and the "message from my number" button.

The cost rule that must not be broken: generate the brief **once**, when the bot hands off, and store it on the lead. Never generate it when the page is opened. A summary that regenerates on every page view is a language-model call every time any rep glances at any lead — which is precisely the runaway API bill this whole feature set is meant to prevent. Offer an explicit "refresh brief" button for the rare case where it is wanted.

Half the data already exists: the bot's notes accumulate on the lead record today. What is missing is a real summary (the existing panel shows only the single most recent note) and the panel being on the lead page at all.

Part 6 — The order to build in

Stage	What	Why this order
Stage 0 immediate	Close the open AI endpoint (8). Fix STOP-after-handoff (3). Remove the "DCGI approved" fallback text (13). Make the Health tab and Knowledge tab say "not connected yet" until they are (1, 2).	Four small changes. Two are costing money or breaking policy right now; two stop the product from telling the user something untrue. None depends on anything else.
Stage 1 the foundation	One send gateway (1.5). Phone-to-party resolution (Part 3, Principle 1). Real number health (1.1). Opt-in / opt-out done properly (1.4). Rep read-only default + owner override (5B).	Everything after this is cheaper and safer because of it. The rep permission rule belongs here because it is a send-path decision, and the send path is being rebuilt anyway.
Stage 2 control & honest numbers	The capability switchboard (5A) and the contacted-tick bot cutoff (5B). Real broadcast reporting and paging (1.2). Template status and quality webhooks (7). Cost meter (2.5). Rate and tier awareness (11).	The switchboard needs the gateway underneath it to be enforceable in both directions. The contacted-tick pays for itself immediately in AI cost, and pairs naturally with the cost meter that proves it worked.
Stage 3 the chore work	Template autopilot (2.1), language fan-out (2.2), order lifecycle messages (2.3), buttons (2.4), follow-up sequences (2.6), knowledge base wired (1.3), merged lead ↔ WhatsApp view (5B).	This is the stage the original brief asked for. It is deliberately not first — automating template creation on top of a send path that ignores opt-outs would industrialise the problem.
Stage 4 the product	Bot serves existing customers (3.1), ambient order capture (3.2), payment intimation (3.3), voice (3.4), conversation intelligence (3.5), rep dashboard (3.6).	Each one depends on the identity resolution built in Stage 1, and each one arrives with its own switch from Stage 2 already in place.
Stage 5 the moat	Ban-risk forecasting (4.1), self-optimising templates (4.2), cashflow-aware follow-up (4.3), conversation memory (4.4), ten-minute onboarding (4.5), compliance pack (4.6).	All of these are cheap extensions of data the earlier stages already collect. None is buildable before them.

The closing judgement

The hard part is done and done well. Building a reliable, secure, always-on WhatsApp pipeline with a bot that behaves itself is the part most teams get wrong, and this one is careful in exactly the places that matter — signatures, duplicates, races, fallbacks, and never quoting a rate it should not know.

What is missing is the discipline of only showing what we actually measured, and the compliance edges that protect the client's WhatsApp number. Both are days of work, not months. **Do Stage 0 and Stage 1 before onboarding a second client** — after that, the automation originally asked for sits on a foundation that can carry it.

The owner controls in Part 5 change the shape of the product in a good way: they turn WhatsApp from a fixed feature into something each company tunes against its own strategy — how much to serve on WhatsApp, how much to reserve for their app, and how much rope to give a rep. That is a far better thing to sell than a bot with no dial on it.

Sources for policy and pricing claims: Meta for Developers — Template fundamentals; Template categorization; WhatsApp Commerce Policy; Meta healthcare-sector commerce restrictions; Marketing Messages Lite (MM Lite) frequency-cap documentation; India 2026 per-message pricing schedules. All code findings were verified by direct reading of the repository on 16 August 2026; no code was modified.